

APIS Y SOCKETS

Resumen — REST, SOAP y comunicación mediante sockets

Resumen esquemático

Basado en el fastbook original de Jaime Morales (Fastbook 04)

Contenido

1. APIs: concepto y tipos	3
2. REST	3
3. SOAP	3
4. Comparativa: REST frente a SOAP	3
5. Sockets	3

1. APIs: concepto y tipos

- ▶ Una **API** (Application Programming Interface) es un módulo de software que permite que las aplicaciones se comuniquen entre sí y compartan información y funcionalidades.
- ▶ Según su acceso: **privadas** (solo personal autorizado, ej. un banco), **públicas** (abiertas a cualquier desarrollador, sin validación) y **de entorno local** (solo accesibles desde una red interna).
- ▶ **API** vs. **endpoint**: la API es la interfaz genérica que ofrece una o varias funcionalidades; el endpoint es la URL concreta a la que se llama para ejecutar una de ellas (request → response).

2. REST

- ▶ **REST** no es un protocolo ni un estándar: es un conjunto de restricciones (constraints) que definen una arquitectura, estableciendo un contrato entre consumidor y productor.
- ▶ Formatos de envío de datos: **JSON** (el más usado), HTML, XLT, Python, PHP, o sin formato.
- ▶ Para que una API sea REST debe cumplir: arquitectura **cliente-servidor**; ser **stateless** (sin estado, cada petición es independiente); ser **cacheable**; tener una **interfaz uniforme** (recursos identificables, manipulables desde su representación, mensajes autodescriptivos e hipermedios/HATEOAS); un **sistema en capas** invisible para el cliente; y, opcionalmente, **código bajo demanda**.
- ▶ Herramientas de documentación: **Swagger** (contrato interactivo vía JSON/YAML), **RAML** (lenguaje de modelado) y **OpenAPI** (estándar nacido de Swagger).

3. SOAP

- ▶ **SOAP** (Simple Object Access Protocol) surgió en los 90 para la comunicación cliente-servidor, funcionando sobre HTTP, HTTPS, FTP o TCP.
- ▶ Se basa en **XML**: toda solicitud es un documento XML bien formado, envuelto en un **envelope** (sobre).
- ▶ Su mensaje se estructura en **Header** (metadatos opcionales, ej. tipo de cifrado) y **Body** (los datos reales del mensaje).

4. Comparativa: REST frente a SOAP

- ▶ **Diseño**: SOAP expone la operación; REST expone los datos.
- ▶ **Transporte**: SOAP funciona sobre cualquier protocolo (HTTP, FTP, TCP...); REST solo sobre HTTP/HTTPS.
- ▶ **Formato**: SOAP solo XML; REST admite XML, JSON, texto plano o HTML.
- ▶ **Rendimiento y escalabilidad**: SOAP es más lento y difícil de escalar (mantiene estado); REST es más rápido (mensajes pequeños, caché) y fácil de escalar (sin estado).
- ▶ **Caso de uso**: SOAP encaja en apps antiguas y APIs privadas; REST, en apps modernas y APIs públicas.

5. Sockets

- ▶ Un **socket** es el punto final de una conexión: un elemento generado por el sistema operativo que permite enviar y recibir datos entre procesos, incluso entre máquinas distintas.
- ▶ Ventajas: intercambio de datos fiable y ordenado, aprovechando además la funcionalidad de las APIs. Desventajas: dependencia de la respuesta del servidor, y en sockets no orientados a conexión (UDP) no se garantiza el orden de llegada.
- ▶ Tipos principales: **SOCK_DGRAM** (mensajes cortos sin conexión, sin garantía de orden), **SOCK_STREAM** (flujo bidireccional fiable y ordenado, base de TCP/IP), **SOCK_SEQPACKET** (secuenciado, fiable, sin duplicados) y **SOCK_CONN_DGRAM** (datagramas orientados a conexión, secuenciados pero sin fiabilidad total).

★ Ideas clave para recordar

- ▶ Las APIs REST son más actuales, ágiles y consumen menos recursos: son el futuro predominante del desarrollo de interfaces web.
- ▶ SOAP sigue siendo relevante en sistemas antiguos o entornos que exigen contratos estrictos.
- ▶ Los sockets son canales de intercomunicación entre procesos, tanto en local como en red.

Repasa este resumen antes de hacer el cuaderno de ejercicios correspondiente